

Sistemas Operativos II - Comparación entre gRPC, REST y GraphQL

Martín Errea

Universidad CAECE,
Mar del Plata, Buenos Aires, Argentina.
{erreamartin65@gmail.com}

Abstract. La arquitectura de microservicios se ha convertido en el paradigma de diseño para crear sistemas de software escalables y mantenibles. La selección del protocolo de comunicación adecuado en los microservicios es fundamental para lograr un rendimiento óptimo del sistema. Este estudio compara el rendimiento de tres protocolos de API de uso común: REST, GraphQL y gRPC, en arquitecturas de microservicios. Se establecieron tres microservicios implementados en tres contenedores, cada uno con una base de datos Redis y MySQL. El rendimiento de estos protocolos se evaluó mediante cuatro métricas clave: tiempo de respuesta y utilización de CPU, tamaño de payload transferido y throughput sostenido. Se realizaron dos tipos distintos de recuperación de datos: obtención de datos planos y datos anidados, con una cantidad de solicitudes que varía entre 100 y 2000. Los resultados experimentales indican que gRPC presenta el menor tamaño de payload con una reducción de entre el 53% y el 64% frente a REST y GraphQL, mientras que en tiempo de respuesta y throughput bruto REST resulta el más rápido, seguido por GraphQL y gRPC. Además GraphQL muestra una mayor utilización de CPU en comparación con gRPC y REST. Los resultados aportan información valiosa para desarrolladores y arquitectos que buscan optimizar los protocolos de comunicación de sus microservicios para casos de uso y cargas de trabajo específicas.

Keywords: Microservicios · API · gRPC · REST · GraphQL

1 Introducción

El desarrollo de software mediante arquitectura de microservicios ha transformado la forma en que diseñamos aplicaciones. Esta arquitectura propone la descomposición de aplicaciones complejas en microservicios más pequeños e independientes. Cada microservicio tiene tareas específicas que pueden gestionarse y modificarse sin afectar a los demás componentes, lo que permite a los equipos de desarrollo concentrarse en aspectos concretos de la aplicación, mejorando la escalabilidad, la velocidad de cambio y el aislamiento de fallas [1].

En la comunicación entre microservicios, dos de los protocolos más utilizados son REST (*Representational State Transfer*) y GraphQL (*Graph Query Language*). REST ha sido uno de los métodos de intercambio de datos más ampliamente adoptados, basado en múltiples endpoints para acceder y manipular

datos. Aunque REST sigue siendo popular, presenta ciertas desventajas, como el *over-fetching* o *under-fetching* de datos, donde la información recuperada puede exceder o no alcanzar las necesidades reales. GraphQL surge como una alternativa atractiva para abordar estas limitaciones, permitiendo que los clientes especifiquen exactamente los datos que necesitan [2, 3], superando así el problema de ineficiencia de REST y otorgando mayor control a los desarrolladores.

Además de REST y GraphQL, otro método de intercambio de datos que gana relevancia actualmente es gRPC (*Google Remote Procedure Call*). gRPC ofrece un enfoque eficiente y versátil para la comunicación entre servicios distribuidos. A diferencia de REST y GraphQL, que utilizan el protocolo HTTP/1, gRPC emplea HTTP/2 y soporta transmisión de datos por flujo (*streaming*). gRPC simplifica las llamadas a procedimientos remotos entre distintos lenguajes de programación, proporcionando mayor rendimiento y velocidad en la comunicación entre microservicios [4].

Este estudio tiene como objetivo evaluar y comparar el rendimiento de REST, gRPC y GraphQL para el intercambio de datos dentro de un sistema de microservicios, tanto en la obtención de datos planos como anidados. El análisis incluye métricas de rendimiento clave: tiempo de respuesta, utilización de CPU, tamaño de payload transferido por la red y throughput sostenido bajo carga, complementadas con la tasa de error observada en cada combinación de protocolo, modalidad y nivel de carga. Con esta evaluación, se busca asistir a desarrolladores y organizaciones en la toma de decisiones informadas al diseñar e implementar sistemas basados en microservicios.

2 Trabajos Relacionados

Existen numerosos estudios que comparan el rendimiento de REST y GraphQL. Vohra y Manuaba [5] describen el rendimiento de REST y GraphQL utilizando los gateways de API Ocelot y Hot Chocolate en operaciones de escritura y lectura de datos. Vadlamini y et al. [6] señalan las ventajas y desventajas de REST y GraphQL, concluyendo que GraphQL es la opción preferida cuando los datos cambian con frecuencia y requieren optimización de recursos. Lawi et al. [7] señalan que REST es la opción adecuada para el intercambio de datos en situaciones donde se accede a información de manera constante.

Marganski y Pánczyk [8] y Mikula y Dzienkowski [9] comparan REST y GraphQL para el diseño de APIs web, evaluando tiempos de respuesta y tamaños de datos usando aplicaciones NodeJS con operaciones CRUD sobre MongoDB. No se observaron diferencias significativas para pocas consultas; GraphQL superó a REST con cargas elevadas y datos pequeños, mientras que REST fue mejor con datos de mayor volumen. França y da Silva [10] evaluaron el rendimiento de REST y GraphQL al consultar objetos anidados, y los resultados indicaron que GraphQL superó a REST en la mayoría de los escenarios. Brito et al. [11] migraron siete sistemas de APIs REST a GraphQL, encontrando que GraphQL puede reducir significativamente el tamaño de los documentos JSON retornados por las APIs REST, con una reducción del 94%.

En cuanto a estudios sobre el rendimiento de gRPC, Vo [12] explica los microservicios y gRPC, cubriendo su funcionamiento, implementaciones, limitaciones y aplicaciones. Stefanic [13] explora la arquitectura de microservicios y sus métodos de comunicación, principalmente REST API y gRPC, evaluando los pros y contras de ambos enfoques. Rebrosova [14] estudia el potencial de gRPC para mejorar la entrega de contenidos, utilizando la metodología GQM (*Goal Question Metric*), con hallazgos que indican un rendimiento excepcional de gRPC en escenarios con aplicaciones móviles o IoT como clientes. Nieman y Sajal [15] analizan el balanceo de carga en microservicios gRPC dentro de Kubernetes usando Golang.

Este estudio se enfoca en la comparación del rendimiento de REST, GraphQL y gRPC en entornos de microservicios, aportando información sobre sus ventajas y desventajas respectivas y revelando qué protocolos de comunicación operan eficientemente en distintos escenarios y cargas de trabajo.

3 Protocolos de Interfaz de Programación de Aplicaciones (API)

Los protocolos de API son conjuntos de reglas, convenciones y estándares que facilitan la comunicación e interacción entre diversos programas y sistemas de software. Estos protocolos definen la estructura y el formato de las solicitudes y respuestas, así como los métodos y reglas para la comunicación. Los protocolos de API más utilizados son REST, GraphQL y gRPC.

3.1 REST (Representational State Transfer)

REST es una arquitectura de desarrollo de API que proporciona comunicación cliente-servidor a través del protocolo HTTP. Fue introducida por primera vez por Roy Fielding en el año 2000 como parte de su disertación doctoral en la Universidad de California [16]. REST utiliza el protocolo HTTP/1.1 para enviar datos desde los clientes hacia los servidores. En sistemas que utilizan REST, cada servicio suele tener un endpoint específico para interactuar y intercambiar datos. REST admite varios métodos, como GET, POST, PUT y DELETE, así como diferentes formatos de presentación de datos, como JSON y XML. JSON se utiliza con mayor frecuencia debido a su simplicidad y eficiencia.

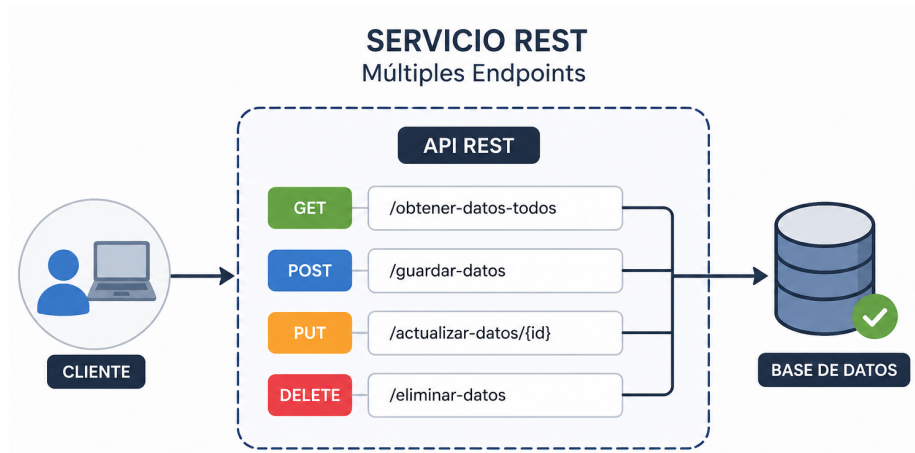


Fig. 1. Modelo REST.

3.2 GraphQL

GraphQL, un lenguaje de consulta para APIs, fue creado por Facebook para la comunicación entre clientes y servidores [17]. El cliente solicita los datos que necesita mediante una consulta, y el servidor devuelve una respuesta acorde a dicha solicitud. GraphQL ofrece una solución alternativa a REST y permite a los desarrolladores solicitar datos específicos de manera más eficiente y flexible. Surgió para satisfacer las necesidades de Facebook en el manejo de datos complejos y para superar los problemas de REST, como el *over-fetching* o *under-fetching* de datos. Una de las principales ventajas de GraphQL es su flexibilidad: los clientes pueden solicitar múltiples fuentes de datos en una única solicitud, reduciendo el número de peticiones necesarias. Además, los clientes pueden validar sus consultas mediante tipos claramente definidos antes de enviarlas al servidor.

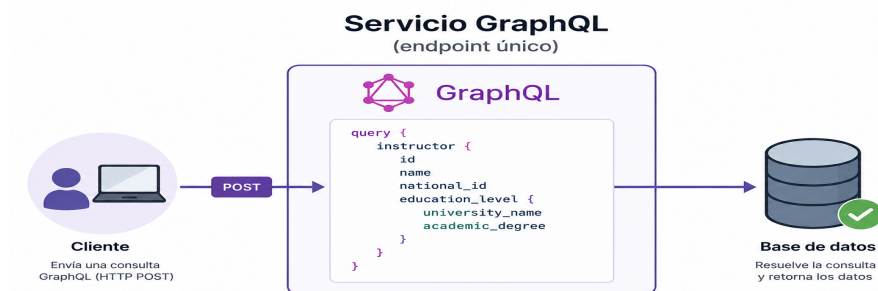


Fig. 2. Modelo GraphQL.

3.3 gRPC (Google Remote Procedure Call)

gRPC [18] es un framework de código abierto y alto rendimiento para construir sistemas distribuidos eficientes y microservicios. Fue desarrollado por Google y diseñado para posibilitar la comunicación entre aplicaciones y servicios de manera agnóstica al lenguaje y a la plataforma. gRPC permite a las aplicaciones definir sus métodos de servicio y estructuras de datos utilizando *Protocol Buffers* (protobufs), un lenguaje de definición de interfaces neutral al lenguaje. A partir de estas definiciones, genera código de cliente y servidor en múltiples lenguajes de programación. La comunicación se realiza usando HTTP/2, aprovechando características como el streaming bidireccional, la multiplexación y la serialización eficiente, lo que lo convierte en un framework de alto rendimiento para sistemas distribuidos y APIs agnósticas al lenguaje.



Fig. 3. Modelo gRPC.

4 Diseño del Sistema

El experimento se ejecutó sobre tres instancias EC2 `t3.micro` en la región `sa-east-1` de AWS, una por protocolo, cada una con su propia base MySQL y su propio Redis (Sección 4.1). El cliente de carga corrió desde una máquina externa a AWS, generando tandas de 100 a 2000 solicitudes en las modalidades *flat* y *nested*, por cinco repeticiones por combinación.

(Tabla 1, Tabla 2, Tabla 3, Tabla 4).

4.1 Tamaño de respuesta

El resultado más limpio del experimento es el tamaño de los mensajes intercambiados (Tabla 1). A diferencia de la latencia, este valor no depende de las condiciones de red ni de la carga: es una propiedad estática del protocolo de serialización para un conjunto de datos fijo. gRPC produce mensajes de 42 bytes en modalidad *flat* y 100 bytes en modalidad *nested*, frente a 89 y 217 de REST y 114 y 242 de bytes de GraphQL. Esto representa una reducción de entre el 53% y el 63% respecto a REST, y de entre el 59% y el 63% respecto a GraphQL, consistente con la ventaja teórica de la serialización binaria de Protocol Buffers sobre JSON documentada en la Sección 3.3. Vale notar que GraphQL retorna más bytes que REST incluso en la modalidad *flat*, lo cual reflejaría el envoltorio (*wrapper*) `{"data":{"instructor":{...}}}` que agrega la capa de protocolo.

Table 1. Tamaño promedio de respuesta por protocolo y modalidad (bytes)

Protocolo	flat	nested
REST	89	217
GraphQL	114	242
gRPC	42	100

4.2 Uso de CPU

El uso de CPU (Tabla 2) confirma el orden predicho por la literatura (Sección 2) de forma contundente: GraphQL es el protocolo más costoso en términos de cómputo (16.4% promedio), seguido por REST (10.5%), con gRPC claramente como el más eficiente (5.0%). Este orden se mantiene en todas las cargas y en ambas modalidades sin ninguna excepción. Al estar cada protocolo aislado en su propia instancia EC2, sin contención con los otros dos, este resultado puede atribuirse con confianza al costo de cómputo propio de cada protocolo: GraphQL paga la resolución de su árbol de tipos en cada solicitud más el parseo de la consulta entrante; REST paga la serialización/deserialización de JSON; y gRPC amortiza ambos costos mediante la serialización binaria de Protocol Buffers, coherente con la ventaja que ya se observó en el tamaño de payload.

Table 2. Uso de CPU promedio (%) – promediado sobre todas las cargas (100–2000 solicitudes, 5 repeticiones)

Protocolo	flat (%)	nested (%)	promedio global (%)
REST	10.67	10.46	10.56
GraphQL	16.11	16.67	16.39
gRPC	5.41	4.56	4.99

4.3 Latencia

La latencia observada está dominada por el viaje de red entre el cliente y la región **sa-east-1**: incluso en el mejor caso de cada protocolo (**min_latency_ms**), los valores no bajan de 39.63 ms en REST, 40.87 ms en GraphQL y 45.01 ms en gRPC. Este piso diferencial de aproximadamente 5 ms a favor de REST y GraphQL respecto a gRPC es estable en la totalidad de las 28 filas del experimento, lo que apunta a que la instancia gRPC quedó ubicada en una zona de disponibilidad distinta dentro de la región, no a una diferencia intrínseca del protocolo.

Si se normaliza la latencia restando el piso mínimo de cada protocolo (para aislar el sobre costo de procesamiento por encima de la red), REST añade aproximadamente 2.5 ms, GraphQL aproximadamente 3.8 ms, y gRPC aproximadamente 3.0 ms, REST y gRPC quedan prácticamente empatados, y GraphQL mantiene el mayor sobre costo de los tres, consistente con sus resultados de CPU y payload.

Un segundo fenómeno visible en los datos es el efecto de calentamiento de caché. La desviación estándar de latencia cae abruptamente a partir de 200–300 solicitudes: en REST *flat* pasa de 6.86 ms con 100 solicitudes a 0.34 ms con 300 y se mantiene por debajo de 1 ms hasta 2000. Esto se confirma en el detalle por repetición (Anexo A.2, donde la primera repetición de cada combinación es sistemáticamente la más lenta (la primera ola de solicitudes puebla la caché Redis y establece las conexiones a MySQL), mientras que las repeticiones 2 a 5 son estables y consistentes. A partir de 300 solicitudes, el sistema opera completamente en régimen de caché caliente, y la variabilidad residual en ese régimen es inferior a 1 ms.

Table 3. Tiempo de respuesta promedio (ms) – modalidad *flat*

Protocolo	100	200	300	400	500	1000	2000
REST	46.52	45.21	42.82	42.32	42.73	42.57	42.88
GraphQL	51.06	45.10	44.68	44.59	44.40	44.08	44.04
gRPC	54.67	50.11	48.63	47.90	47.77	48.18	47.78

Table 4. Tiempo de respuesta promedio (ms) – modalidad *nested*

Protocolo	100	200	300	400	500	1000	2000
REST	43.58	43.08	43.90	42.44	42.87	42.63	42.80
GraphQL	46.79	45.40	44.88	44.86	45.21	44.84	44.90
gRPC	53.39	51.03	48.74	47.93	48.08	48.08	47.94

4.4 Throughput

El throughput observado (Tabla 5) fue similar entre los tres protocolos, oscilando entre 200 y 250 solicitudes por segundo en la mayoría de las combinaciones, sin una tendencia de crecimiento ni de degradación clara a medida que la carga total aumenta de 100 a 2000 solicitudes.

Table 5. Throughput promedio (solicitudes/segundo) por protocolo y modalidad.

Protocolo	flat (req/s)	nested (req/s)
REST	240.2	248.0
GraphQL	226.0	229.9
gRPC	207.7	209.4

Esta equivalencia relativa entre protocolos es esperable dado el diseño del experimento: con una concurrencia fija de 10 solicitudes simultáneas y un piso de red de entre 40 y 47 ms, el throughput teórico máximo queda acotado por concurrencia/latencia $\approx 10/0.043 \approx 233$ req/s para REST y GraphQL, y $10/0.048 \approx 208$ req/s para gRPC, valores muy cercanos a los observados. En otras palabras, bajo esta metodología el throughput no mide la capacidad de cada protocolo de manejar concurrencia, sino que es una consecuencia aritmética de la latencia y de un parámetro de concurrencia que se mantuvo constante en las 210 repeticiones.

4.5 Comparación con las hipótesis de la literatura

La Tabla 6 y la Tabla 7 contrastan, para cada antecedente citado en la Sección 2, la hipótesis o hallazgo reportado y si el presente experimento lo confirma, lo matiza o no permite evaluarlo con las condiciones probadas.

Table 6. Contraste con la literatura (Parte I)

Antecedente	Hipótesis / hallazgo	Resultado obtenido
Vadlamani et al. [6]	GraphQL es preferible cuando los datos cambian con frecuencia y se requiere optimización de recursos.	No confirmado. GraphQL presentó el mayor consumo de CPU (16.4% frente a 10.5% de REST), resultando el protocolo menos eficiente en las condiciones evaluadas.
Lawi et al. [7]	REST es adecuado para accesos constantes a la información.	Parcialmente confirmado. REST obtuvo el menor consumo de CPU entre los protocolos basados en JSON y la menor latencia absoluta.
Marganski y Pánczyk [8]; Mikula y Dzienkowski [9]	GraphQL supera a REST con cargas elevadas y datos pequeños.	No confirmado. Hasta 2000 solicitudes, REST mantuvo menor latencia y menor uso de CPU que GraphQL.
França y da Silva [10]	GraphQL supera a REST para consultas anidadas.	No confirmado. REST obtuvo menor latencia en todas las cargas evaluadas para consultas <i>nested</i> .

Table 7. Contraste con la literatura (Parte II)

Antecedente	Hipótesis / hallazgo	Resultado obtenido
Brito et al. [11]	GraphQL reduce el tamaño de los documentos JSON hasta un 94%.	Dirección opuesta. En este trabajo GraphQL produjo respuestas mayores que REST. Debido al tamaño reducido del esquema utilizado (4-6 campos), el costo fijo del envoltorio GraphQL superó cualquier ahorro por evitar <i>over-fetching</i> .
Rebrosova [14]; Vo [12]; Stefanic [13]	gRPC ofrece mayor rendimiento gracias a HTTP/2 y Protocol Buffers.	Parcialmente confirmado. gRPC obtuvo el menor consumo de CPU y el menor tamaño de payload. La mayor latencia absoluta se explicó por un piso de red aproximadamente 5 ms superior.

5 Conclusiones

Este trabajo implementó y comparó tres microservicios funcionalmente equivalentes, REST, GraphQL y gRPC, desplegados de forma aislada en instancias EC2 independientes de AWS, midiendo latencia, uso de CPU, tamaño de respuesta y throughput bajo cargas de 100 a 2000 solicitudes en dos modalidades de recuperación de datos, y contrastando los resultados contra las hipótesis de nueve antecedentes (Sección 5.2).

El hallazgo más sólido, por no depender de las condiciones de red ni de la carga, es el **tamaño de payload**: gRPC produce mensajes entre un 53% y un 63% más chicos que REST y GraphQL, confirmando la ventaja de Protocol Buffers sobre JSON (Sección 3.3) reportada por Rebrosova, Vo y Stefanic. Sin embargo, contrario a lo reportado por Brito et al. para GraphQL, en este esquema chico GraphQL terminó *augmentando* el tamaño de respuesta respecto a REST, lo que matiza esa hipótesis de la literatura: la reducción de payload que ofrece GraphQL depende del tamaño del esquema y de cuánto *over-fetching* evita, no es una propiedad universal del protocolo.

El segundo hallazgo robusto es el **uso de CPU**: GraphQL consume aproximadamente el triple de CPU que gRPC y un 55% más que REST, en un orden que se mantiene sin excepción en las 14 combinaciones de carga y modalidad. Esto contradice directamente la hipótesis de Vadlamani et al. de que GraphQL ofrece mejor optimización de recursos, y confirma en cambio la expectativa teórica de que la resolución de un árbol de tipos en cada solicitud tiene un costo de cómputo medible y consistente.

En **latencia**, REST resultó el más rápido en términos absolutos en las catorce combinaciones probadas, contradiciendo a Marganski y Pánczyk, Mikula y Dzienkowski, y França y da Silva, que reportan ventajas de GraphQL bajo carga o en consultas anidadas. Al analizar la composición de esa latencia, sin embargo, se encontró que gran parte de la diferencia entre gRPC y los otros dos protocolos corresponde a un piso de red estructural (aproximadamente 5 ms mayor, estable en las 14 combinaciones) más que a un costo de procesamiento del protocolo; normalizando por ese piso, gRPC y REST quedan prácticamente

equivalentes en sobre costo de procesamiento, lo cual sí es consistente con la hipótesis de Rebrosova, Vo y Stefanic sobre la eficiencia de gRPC.

El **throughput** no permitió diferenciar a los protocolos entre sí (200–250 req/s en los tres casos): bajo el diseño experimental usado, con concurrencia fija en 10 solicitudes, el throughput quedó determinado aritméticamente por la latencia y el nivel de concurrencia, no por una capacidad diferencial de cada protocolo para sostener carga simultánea. Ninguno de los antecedentes citados aborda explícitamente esta limitación, por lo que se considera un aporte metodológico de este trabajo: el throughput medido bajo concurrencia constante no es, por sí solo, evidencia de escalabilidad de un protocolo.

En síntesis, de las hipótesis revisadas en las Tablas 6 y 7, este trabajo confirma las que atribuyen a gRPC eficiencia en serialización y CPU, y refuta o matiza las que atribuyen a GraphQL ventajas de latencia o de tamaño de payload bajo, esquemas pequeños, una discrepancia que se interpreta como dependiente del tamaño del esquema de datos, más que como una contradicción de la literatura.

Como limitaciones se identifican: la falta de control explícito sobre la zona de disponibilidad de las tres instancias EC2; la ejecución del cliente de carga desde fuera de la región de AWS; la concurrencia fija en 10 solicitudes, que impide observar diferencias de escalabilidad entre protocolos en el throughput.

Referencias

1. I. K. Aksakalli, T. Çelik, A. B. Can, and B. Tekinerdogan, “Deployment and communication patterns in microservice architectures: A systematic literature review,” *Journal of Systems and Software*, vol. 180, p. 111014, 2021.
2. G. S. M. Diyasa, G. S. Budiwitjaksono, H. A. Ma’rufi, and I. A. W. Sampurno, “Comparative analysis of rest and graphql technology on nodejs-based api development,” in *Nusantara Science and Technology Proceedings*, pp. 43–52, 2021.
3. M. Vesic and N. Kojic, “Comparative analysis of web application performance in case of using rest versus graphql,” in *Proceedings of the Fourth International Scientific Conference ITEMA*, pp. 17–24, 2020.
4. Y. Lee and Y. Liu, “Using refactoring to migrate rest applications to grpc,” in *Proceedings of the 2022 ACM Southeast Conference*, pp. 219–223, 2022.
5. N. Vohra and I. B. K. Manuaba, “Implementation of rest api vs graphql in microservice architecture,” in *2022 International Conference on Information Management and Technology (ICIMTech)*, pp. 45–50, 2022.
6. S. L. Vadlamani, B. Emdon, J. Arts, and O. Baysal, “Can graphql replace rest? a study of their efficiency and viability,” in *2021 IEEE/ACM 8th International Workshop on Software Engineering Research and Industrial Practice (SER&IP)*, pp. 10–17, 2021.
7. A. Lawi, B. L. Panggabean, and T. Yoshida, “Evaluating graphql and rest api services performance in a massive and intensive accessible information system,” *Computers*, vol. 10, no. 11, p. 138, 2021.
8. P. Marganski and B. Pánczyk, “Rest and graphql comparative analysis,” *Journal of Computer Sciences Institute*, vol. 19, pp. 89–94, 2021.
9. M. Mikula and M. Dzienkowski, “Comparison of rest and graphql web technology performance,” *Journal of Computer Sciences Institute*, vol. 16, pp. 309–316, 2020.

10. M. D. C. França and E. da Silva, “Performance evaluation of rest and graphql apis searching nested objects,” *Anais do Computer on the Beach*, vol. 11, no. 1, pp. 237–244, 2020.
11. G. Brito, T. Mombach, and M. T. Valente, “Migrating to graphql: A practical assessment,” in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 140–150, 2019.
12. H. Vo, “Applying microservice architecture with modern grpc api to scale up large and complex application,” 2021.
13. M. Stefanic, “Developing the guidelines for migration from restful microservices to grpc.” Masaryk University, Faculty of Informatics, 2021.
14. B. P. Rebrosova, “grpc layer for content delivery in kenticore kontent.” Master’s thesis, Masaryk University, 2021.
15. K. Nieman and S. Sajal, “A comparative analysis on load balancing and grpc microservices in kubernetes,” in *2023 Intermountain Engineering, Technology and Computing (IETC)*, pp. 322–327, 2023.
16. R. T. Fielding, *Architectural styles and the design of network-based software architectures*. PhD thesis, University of California, Irvine, 2000.
17. The GraphQL Foundation, “GraphQL.” <https://graphql.org/>, 2015.
18. Louis Ryan (Google), “grpc.” <https://grpc.io>, 2015.

A Anexo Técnico

A.1 Evidencia de despliegue

La Figura 4 documenta, respectivamente, las tres instancias EC2 en estado *Running* en la consola de AWS, y una corrida del cliente de carga contra los tres servicios desplegados.

Nombre	ID de la instancia	Estado de la instancia	Tipo de instancia	Comprobación de estado	Zona de disponibilidad	DNS de IPv4 pública	Dirección IP pública	IP elástica	Direcciones de salida	Monitores	Nombre del grupo de seguridad	Nombre de clave	Fecha de lanzamiento
so2-bench-grpc	i-08b7b9b5d0c35716a8	En ejecución	t3.micro	1/1 comprobado	sa-east-1b	ec2-56-125-151-206.sa-east-1.amazonaws.com	56.125.151.206	-	-	disabled	so2-bench-rg	so2-bench-key	2024/09/09 16:17
so2-bench-rest	i-0d279b6d2c6a8d6a8	En ejecución	t3.micro	1/1 comprobado	sa-east-1b	ec2-56-125-48-21.sa-east-1.amazonaws.com	56.125.48.21	-	-	disabled	so2-bench-rg	so2-bench-key	2024/09/09 16:17
so2-bench-graphql	i-0f029b3a0d19b0a0a8	En ejecución	t3.micro	1/1 comprobado	sa-east-1b	ec2-56-124-90-231.sa-east-1.amazonaws.com	56.124.90.231	-	-	disabled	so2-bench-rg	so2-bench-key	2024/09/09 16:17

Fig. 4. Las tres instancias EC2 (*so2-bench-rest*, *so2-bench-graphql*, *so2-bench-grpc*) en estado *Running*, región *sa-east-1*.

A.2 Conjunto de datos crudo

La Tabla 8 reproduce el archivo `results-cloud.csv` completo (resultados agregados sobre las 5 repeticiones por combinación), incluyendo columnas no discutidas en el cuerpo del informe (desviación estándar de latencia y de CPU, cantidad y tasa de errores). El archivo `results-cloud-detail.csv`, con las 210 mediciones individuales por repetición, se adjunta como archivo separado por su extensión ([[enlace al repositorio](#) o [adjuntar el .csv](#)]).

Table 8. Resultados agregados completos (5 repeticiones por fila) – protocolo REST

Modo	Carga	avg ms	stddev ms	p95 ms	CPU %	bytes	req/s
flat	100	46.52	6.86	51.03	2.12	89	205.93
flat	200	45.21	2.99	59.65	8.36	89	216.26
flat	300	42.82	0.34	45.52	4.87	89	231.37
flat	400	42.32	0.20	44.01	10.53	89	279.68
flat	500	42.73	0.55	44.88	15.34	89	231.78
flat	1000	42.57	0.33	44.51	15.47	89	251.32
flat	2000	42.88	0.78	45.29	17.98	89	240.64
nested	100	43.58	1.72	47.28	1.54	217	223.21
nested	200	43.08	0.55	46.69	4.83	217	344.71
nested	300	43.90	1.52	48.53	9.04	217	224.85
nested	400	42.44	0.28	44.25	11.37	217	232.75
nested	500	42.87	0.23	45.13	14.46	217	267.81
nested	1000	42.63	0.20	44.65	15.90	217	251.56
nested	2000	42.80	0.41	44.93	16.10	217	241.48

Table 9. Resultados agregados completos (5 repeticiones por fila) – protocolo GraphQL

Modo	Carga	avg ms	stddev ms	p95 ms	CPU %	bytes	req/s
flat	100	51.06	8.49	72.55	13.02	114	188.82
flat	200	45.10	0.39	51.04	11.45	114	244.74
flat	300	44.68	1.07	50.24	8.89	114	222.19
flat	400	44.59	0.43	48.52	17.51	114	222.82
flat	500	44.40	0.71	47.58	21.57	114	233.73
flat	1000	44.08	0.32	47.21	19.14	114	225.80
flat	2000	44.04	0.41	46.71	21.16	114	243.67
nested	100	46.79	2.52	53.46	2.14	242	205.85
nested	200	45.40	0.45	51.35	10.45	242	215.66
nested	300	44.88	0.26	48.79	11.53	242	282.01
nested	400	44.86	0.56	48.97	19.93	242	221.09
nested	500	45.21	0.84	49.11	23.45	242	218.49
nested	1000	44.84	0.51	47.91	25.16	242	238.56
nested	2000	44.90	0.33	48.21	24.00	242	238.74

Table 10. Resultados agregados completos (5 repeticiones por fila) – protocolo gRPC

Modo	Carga	avg ms	stddev ms	p95 ms	CPU %	bytes	req/s
flat	100	54.67	6.67	59.96	6.17	42	181.82
flat	200	50.11	0.76	52.93	3.77	42	199.36
flat	300	48.63	0.98	51.76	5.89	42	205.51
flat	400	47.90	0.83	50.43	5.39	42	248.11
flat	500	47.77	0.39	50.56	4.85	42	209.29
flat	1000	48.18	0.55	52.49	6.25	42	221.61
flat	2000	47.78	0.44	50.74	5.54	42	224.26
nested	100	53.39	5.50	68.45	3.29	100	186.85
nested	200	51.03	0.51	55.66	3.56	100	195.77
nested	300	48.74	0.76	51.35	1.89	100	205.11
nested	400	47.93	0.36	51.15	5.25	100	250.75
nested	500	48.08	0.34	51.20	5.31	100	207.92
nested	1000	48.08	0.47	51.40	6.76	100	222.06
nested	2000	47.94	0.14	50.72	5.89	100	223.32

Table 11. Resultados de benchmark para protocolo **rest**, modo **flat**.

Requests	Rep.	Avg Lat. (ms)	p95 Lat. (ms)	Min Lat. (ms)	Max Lat. (ms)	CPU %	Payload (B)	Errors	Elapsed (ms)	Throughput (rps)
100	1	58.78	151.78	40.61	159.69	10.07	89	0	608	164.47
100	2	43.23	46.81	40.23	49.05	0.01	89	0	441	226.76
100	3	43.52	46.87	40.35	86.56	0.00	89	0	467	214.13
100	4	43.13	47.20	40.63	52.31	0.54	89	0	439	227.79
100	5	43.94	48.63	40.39	91.00	0.00	89	0	473	211.42
200	1	47.04	77.00	40.15	115.82	27.64	89	0	951	210.30
200	2	49.61	66.02	40.84	118.88	0.00	89	0	1026	194.93
200	3	43.04	46.27	40.57	50.35	7.49	89	0	875	228.57
200	4	42.94	44.72	40.18	52.55	0.00	89	0	873	229.10
200	5	43.42	46.44	40.14	86.01	6.65	89	0	899	222.47
300	1	43.21	45.50	40.57	49.55	0.00	89	0	1305	229.89
300	2	42.88	45.44	40.22	48.49	12.55	89	0	1294	231.84
300	3	42.97	46.25	40.35	50.69	0.00	89	0	1315	228.14
300	4	42.75	46.40	40.08	49.93	11.80	89	0	1292	232.20
300	5	42.29	44.06	39.92	46.38	0.00	89	0	1277	234.93
400	1	42.27	43.93	39.90	50.15	15.15	89	0	1717	232.96
400	2	42.17	43.75	39.84	52.53	14.09	89	0	1713	233.51
400	3	42.66	45.43	39.88	88.24	0.00	89	0	1738	230.15
400	4	42.20	43.74	39.79	46.34	9.76	89	0	258	231.39
400	5	42.31	43.98	39.79	86.55	13.64	89	0	1725	231.88
500	1	42.15	43.98	39.92	47.29	17.61	89	0	2132	234.52
500	2	42.22	43.80	40.08	47.10	14.29	89	0	2128	234.96
500	3	42.98	44.60	40.46	88.53	13.06	89	0	2174	229.99
500	4	43.46	46.82	40.70	53.70	18.67	89	0	2202	227.07
500	5	42.83	44.27	40.42	46.78	13.07	89	0	2150	232.56
1000	1	42.45	44.15	39.85	49.44	16.55	89	0	4270	234.19
1000	2	42.60	44.43	40.27	90.60	15.14	89	0	4285	233.37
1000	3	42.36	44.29	39.86	87.73	14.36	89	0	4255	235.02
1000	4	42.32	44.34	39.82	92.62	15.67	89	0	2757	362.71
1000	5	43.12	45.76	39.90	93.46	15.63	89	0	4328	231.05
2000	1	44.21	55.76	39.90	123.24	28.98	89	0	8858	225.78
2000	2	42.61	44.66	39.81	99.95	14.41	89	0	8552	233.86
2000	3	42.17	43.60	39.63	51.68	16.10	89	0	7006	285.47
2000	4	42.62	44.25	40.03	95.42	15.25	89	0	8552	233.86
2000	5	42.80	45.23	39.89	91.15	15.16	89	0	8587	232.91

Table 12. Resultados de benchmark para protocolo **rest**, modo **nested**.

Requests	Rep.	Avg Lat. (ms)	p95 Lat. (ms)	Min Lat. (ms)	Max Lat. (ms)	CPU %	Payload (B)	Errors	Elapsed (ms)	Throughput (rps)
100	1	46.56	70.50	40.29	73.11	2.83	217	0	472	211.86
100	2	42.74	46.37	40.46	47.43	0.00	217	0	438	228.31
100	3	42.34	44.42	40.48	45.60	2.20	217	0	431	232.02
100	4	43.54	47.28	40.20	87.43	0.00	217	0	465	215.05
100	5	42.71	45.63	40.09	46.85	2.66	217	0	434	230.41
200	1	42.16	43.74	40.11	46.26	0.84	217	0	-648	-308.64
200	2	43.37	46.18	40.48	87.36	0.00	217	0	900	222.22
200	3	43.32	47.64	40.36	87.43	13.17	217	0	891	224.47
200	4	43.55	48.90	40.40	58.72	0.00	217	0	880	227.27
200	5	42.99	48.41	40.15	52.17	10.15	217	0	878	227.79
300	1	43.12	45.76	40.18	91.48	0.00	217	0	1321	227.10
300	2	42.77	44.95	40.68	49.03	13.51	217	0	1291	232.38
300	3	42.84	45.16	40.39	50.03	0.00	217	0	1292	232.20
300	4	44.44	49.74	40.53	85.11	31.71	217	0	1359	220.75
300	5	46.34	61.59	40.65	107.44	0.00	217	0	1408	213.07
400	1	42.89	45.07	40.25	51.85	14.79	217	0	1735	230.55
400	2	42.42	44.35	39.85	49.48	14.66	217	0	1708	234.19
400	3	42.43	43.91	39.85	63.57	0.00	217	0	1713	233.51
400	4	42.33	44.04	39.94	85.21	12.22	217	0	1729	231.35
400	5	42.14	43.52	39.89	48.22	15.16	217	0	1708	234.19
500	1	42.91	45.52	40.49	54.27	18.27	217	0	2155	232.02
500	2	42.83	44.58	40.51	88.32	12.00	217	0	700	232.10
500	3	42.51	43.88	40.36	47.76	14.14	217	0	2154	232.13
500	4	43.13	45.12	40.28	91.82	15.29	217	0	2173	230.10
500	5	42.93	46.05	39.94	54.35	12.62	217	0	2153	232.23
1000	1	42.34	43.81	40.06	47.61	15.93	217	0	4250	235.29
1000	2	42.54	44.14	40.12	87.24	14.95	217	0	4279	233.70
1000	3	42.88	45.53	40.33	89.86	14.60	217	0	4309	232.07
1000	4	42.70	44.76	40.23	86.59	17.18	217	0	4290	233.10
1000	5	42.71	44.90	40.15	85.58	16.84	217	0	2748	363.90
2000	1	42.39	43.91	39.92	84.11	14.93	217	0	8501	235.27
2000	2	42.83	44.51	40.03	110.49	14.70	217	0	8598	232.61
2000	3	43.47	48.30	40.18	89.40	20.20	217	0	8709	229.65
2000	4	42.73	44.93	39.91	108.21	15.39	217	0	7058	283.37
2000	5	42.59	44.40	40.06	92.15	15.27	217	0	8545	234.06

Table 13. Resultados de benchmark para protocolo `graphql`, modo `flat`.

Requests	Rep.	Avg Lat. (ms)	p95 Lat. (ms)	Min Lat. (ms)	Max Lat. (ms)	CPU %	Payload (B)	Errors	Elapsed (ms)	Throughput (rps)
100	1	66.08	212.65	42.23	216.32	22.42	114	0	689	145.14
100	2	46.38	57.00	43.14	60.47	0.00	114	0	471	212.31
100	3	49.46	75.15	42.56	93.68	19.12	114	0	525	190.48
100	4	46.55	58.07	42.22	62.87	0.00	114	0	475	210.53
100	5	46.85	55.99	42.31	58.51	23.58	114	0	488	204.92
200	1	45.57	57.74	41.86	67.75	0.00	114	0	920	217.39
200	2	44.86	48.15	41.66	54.73	22.58	114	0	901	221.98
200	3	45.01	51.04	41.35	57.58	0.00	114	0	910	219.78
200	4	45.43	51.18	41.66	91.36	23.76	114	0	932	214.59
200	5	44.63	49.71	41.54	62.46	10.93	114	0	423	472.81
300	1	46.57	55.41	41.50	97.37	0.00	114	0	1417	211.71
300	2	44.13	48.43	41.66	54.53	21.88	114	0	1334	224.89
300	3	44.48	48.50	41.08	55.94	0.00	114	0	1339	224.05
300	4	44.25	48.65	41.27	53.04	22.54	114	0	1332	225.23
300	5	43.98	47.10	41.22	55.95	0.00	114	0	1329	225.73
400	1	44.60	47.84	41.48	55.06	24.69	114	0	1794	222.97
400	2	44.77	49.90	41.42	88.32	19.04	114	0	1812	220.75
400	3	45.21	52.74	41.37	117.22	25.24	114	0	1819	219.90
400	4	44.23	47.11	41.39	53.82	18.57	114	0	1775	225.35
400	5	44.15	47.49	41.24	54.66	0.00	114	0	1776	225.23
500	1	44.04	46.59	41.74	90.27	18.18	114	0	2232	224.01
500	2	44.62	47.82	41.73	87.54	27.58	114	0	2260	221.24
500	3	45.54	57.53	41.40	84.44	20.63	114	0	2301	217.30
500	4	43.76	46.29	41.11	55.25	24.45	114	0	2192	228.10
500	5	44.03	45.96	41.04	88.78	17.02	114	0	1711	292.23
1000	1	44.08	47.52	41.20	86.27	19.89	114	0	4443	225.07
1000	2	44.00	46.34	41.23	88.83	18.37	114	0	4425	225.99
1000	3	43.69	45.90	41.01	53.27	18.31	114	0	4379	228.36
1000	4	44.57	48.76	41.40	92.24	19.67	114	0	4488	222.82
1000	5	44.04	46.61	41.06	54.67	19.47	114	0	4408	226.86
2000	1	43.99	46.40	41.36	88.59	19.80	114	0	7309	273.64
2000	2	43.85	46.05	41.12	90.87	19.96	114	0	8789	227.56
2000	3	43.90	46.25	41.24	95.00	20.03	114	0	8806	227.12
2000	4	44.76	51.19	41.21	98.14	26.02	114	0	8958	223.26
2000	5	43.72	45.81	40.87	103.59	20.00	114	0	7177	278.67

Table 14. Resultados de benchmark para protocolo `graphql`, modo `nested`.

Requests	Rep.	Avg Lat. (ms)	p95 Lat. (ms)	Min Lat. (ms)	Max Lat. (ms)	CPU %	Payload (B)	Errors	Elapsed (ms)	Throughput (rps)
100	1	51.20	100.15	41.55	103.58	0.00	242	0	527	189.75
100	2	45.91	52.37	42.28	91.22	5.36	242	0	496	201.61
100	3	45.39	53.23	42.51	56.60	0.00	242	0	457	218.82
100	4	45.06	50.28	41.87	52.97	5.32	242	0	455	219.78
100	5	46.41	52.99	42.32	92.77	0.00	242	0	494	202.43
200	1	45.26	53.03	42.05	59.76	15.03	242	0	910	219.78
200	2	45.40	53.26	42.01	95.24	0.00	242	0	936	213.68
200	3	44.86	48.62	42.09	87.77	15.94	242	0	931	214.82
200	4	46.11	51.85	41.81	94.26	0.00	242	0	947	211.19
200	5	45.35	51.00	41.61	55.38	21.29	242	0	913	219.06
300	1	45.05	48.78	41.75	89.39	0.00	242	0	1388	216.14
300	2	44.51	47.35	41.76	53.56	22.67	242	0	1341	223.71
300	3	45.18	52.44	41.62	56.86	0.00	242	0	1363	220.10
300	4	44.78	48.00	42.07	55.60	11.86	242	0	-152	212.68
300	5	44.89	47.45	42.08	92.54	23.11	242	0	1379	217.55
400	1	44.72	49.98	41.40	90.36	23.57	242	0	1812	220.75
400	2	44.12	45.92	41.70	54.59	0.00	242	0	1780	224.72
400	3	45.70	51.12	41.51	90.10	27.52	242	0	1851	216.10
400	4	44.85	47.58	41.70	54.01	21.52	242	0	1799	222.35
400	5	44.93	48.05	41.64	54.32	27.04	242	0	1804	221.73
500	1	45.24	48.11	41.77	89.49	21.93	242	0	2300	217.39
500	2	44.63	46.76	42.22	88.63	28.04	242	0	2266	220.65
500	3	46.61	56.54	42.32	96.11	22.25	242	0	2348	212.95
500	4	45.06	48.15	42.13	91.07	20.62	242	0	2289	218.44
500	5	44.51	47.46	42.18	54.29	24.42	242	0	2239	223.31
1000	1	44.77	47.91	41.68	88.99	25.03	242	0	4489	222.77
1000	2	45.09	49.68	41.54	98.50	24.69	242	0	2956	338.29
1000	3	44.43	46.97	41.50	86.88	24.20	242	0	4480	223.21
1000	4	45.58	50.14	41.51	135.16	27.78	242	0	4575	218.58
1000	5	44.34	46.31	41.55	89.32	24.09	242	0	4459	224.27
2000	1	44.61	47.73	41.45	91.52	23.97	242	0	8952	223.41
2000	2	44.68	47.74	41.38	89.38	23.20	242	0	7469	267.77
2000	3	45.44	48.76	42.03	100.20	23.43	242	0	9108	219.59
2000	4	44.96	48.61	41.31	96.21	22.78	242	0	9028	221.53
2000	5	44.78	48.27	41.49	87.49	26.61	242	0	7330	272.85

Table 15. Resultados de benchmark para protocolo `grpc`, modo `flat`.

Requests	Rep.	Avg Lat. (ms)	p95 Lat. (ms)	Min Lat. (ms)	Max Lat. (ms)	CPU %	Payload (B)	Errors	Elapsed (ms)	Throughput (rps)
100	1	66.56	183.11	46.00	186.07	11.83	42	0	679	147.28
100	2	52.02	53.91	48.60	54.13	0.00	42	0	521	191.94
100	3	51.72	54.47	47.88	54.63	10.18	42	0	518	193.05
100	4	50.68	52.59	47.20	53.19	0.00	42	0	508	196.85
100	5	52.39	61.95	49.75	62.02	8.86	42	0	524	190.84
200	1	51.22	56.84	46.41	59.58	0.00	42	0	1025	195.12
200	2	49.95	52.23	46.88	53.40	7.18	42	0	1000	200.00
200	3	50.34	52.99	48.40	55.45	0.00	42	0	1008	198.41
200	4	49.92	53.42	46.90	56.33	11.65	42	0	999	200.20
200	5	49.13	51.24	47.49	53.81	0.00	42	0	984	203.25
300	1	49.05	52.67	46.39	54.70	10.30	42	0	1473	203.67
300	2	48.39	50.31	46.12	50.86	0.00	42	0	1452	206.61
300	3	50.12	59.13	45.50	74.84	13.64	42	0	1504	199.47
300	4	47.95	50.59	45.33	52.76	0.00	42	0	1439	208.48
300	5	47.66	50.29	45.25	51.54	5.50	42	0	1431	209.64
400	1	47.95	50.65	46.09	53.26	3.84	42	0	397	1007.56
400	2	47.64	49.77	45.11	50.21	9.35	42	0	1906	209.86
400	3	49.30	56.93	45.13	74.92	4.27	42	0	1972	202.84
400	4	47.18	48.78	45.68	50.92	4.45	42	0	1888	211.86
400	5	47.42	49.47	45.61	50.59	5.05	42	0	1898	210.75
500	1	47.95	50.76	46.12	60.58	4.38	42	0	2399	208.42
500	2	47.41	49.31	45.05	50.09	4.98	42	0	2371	210.88
500	3	48.31	53.00	46.55	56.09	4.43	42	0	2416	206.95
500	4	47.77	50.51	45.74	57.18	5.64	42	0	2389	209.29
500	5	47.39	49.49	45.35	50.25	4.84	42	0	2370	210.97
1000	1	47.98	52.48	45.94	59.96	5.89	42	0	4799	208.38
1000	2	47.62	49.78	45.58	56.25	4.52	42	0	3229	309.69
1000	3	48.93	59.45	46.19	84.82	5.86	42	0	4895	204.29
1000	4	48.56	55.78	45.73	64.46	8.79	42	0	4857	205.89
1000	5	47.81	49.85	45.51	61.16	6.17	42	0	4782	209.12
2000	1	48.54	51.39	45.88	59.82	6.03	42	0	9709	205.99
2000	2	47.51	50.16	46.00	56.64	5.48	42	0	7811	256.05
2000	3	47.48	50.01	45.05	58.28	5.64	42	0	9495	210.64
2000	4	47.67	51.43	45.01	60.11	5.13	42	0	9534	209.78
2000	5	47.70	50.64	45.19	57.93	5.44	42	0	8043	248.66

Table 16. Resultados de benchmark para protocolo `grpc`, modo `nested`.

Requests	Rep.	Avg Lat. (ms)	p95 Lat. (ms)	Min Lat. (ms)	Max Lat. (ms)	CPU %	Payload (B)	Errors	Elapsed (ms)	Throughput (rps)
100	1	55.43	104.70	45.44	106.04	0.00	100	0	555	180.18
100	2	47.75	49.88	45.51	50.29	2.16	100	0	478	209.21
100	3	53.09	68.45	46.73	68.54	0.00	100	0	532	187.97
100	4	61.56	133.98	48.09	134.08	14.29	100	0	617	162.07
100	5	49.11	52.71	46.17	53.13	0.00	100	0	494	202.43
200	1	50.77	54.25	46.55	55.56	4.11	100	0	1017	196.66
200	2	50.89	53.03	47.53	53.88	0.00	100	0	1019	196.27
200	3	50.39	54.97	47.70	56.29	5.25	100	0	1009	198.22
200	4	51.48	63.41	47.05	69.50	0.00	100	0	1030	194.17
200	5	51.61	60.01	47.52	64.95	8.43	100	0	1033	193.61
300	1	49.53	51.79	47.61	52.81	0.00	100	0	1487	201.75
300	2	49.49	53.21	46.76	60.73	4.88	100	0	1485	202.02
300	3	48.68	50.67	45.89	53.51	0.00	100	0	1460	205.48
300	4	47.92	49.54	45.96	50.09	4.56	100	0	1438	208.62
300	5	48.09	50.65	45.99	55.21	0.00	100	0	1443	207.90
400	1	48.55	53.48	46.37	56.00	5.43	100	0	331	208.46
400	2	47.88	50.76	45.93	51.91	6.21	100	0	1915	208.88
400	3	47.78	50.25	46.11	55.75	4.69	100	0	1912	209.21
400	4	47.81	49.87	45.34	60.43	5.33	100	0	1913	209.10
400	5	47.62	51.11	45.77	52.77	4.58	100	0	1905	209.97
500	1	48.03	51.17	45.87	54.84	5.31	100	0	2402	208.16
500	2	47.96	50.58	45.85	55.98	7.09	100	0	2399	208.42
500	3	48.20	50.60	45.99	58.78	4.26	100	0	2411	207.38
500	4	47.65	49.55	46.56	51.58	5.36	100	0	2383	209.82
500	5	48.57	53.57	46.61	59.07	4.55	100	0	2429	205.85
1000	1	47.78	49.51	45.66	59.38	5.06	100	0	4778	209.29
1000	2	47.77	49.79	45.46	56.25	5.38	100	0	3232	309.41
1000	3	47.69	49.45	45.71	54.65	7.25	100	0	4770	209.64
1000	4	48.42	54.45	42.95	68.17	11.49	100	0	4861	205.72
1000	5	48.74	52.48	45.87	69.52	4.62	100	0	4875	205.13
2000	1	48.12	51.41	46.08	62.96	6.30	100	0	9624	207.81
2000	2	47.80	49.88	46.04	63.73	5.30	100	0	7948	251.64
2000	3	47.94	50.49	46.14	60.79	6.14	100	0	9588	208.59
2000	4	48.04	51.58	45.59	58.86	5.92	100	0	9609	208.14
2000	5	47.80	50.42	45.50	56.44	5.76	100	0	8009	249.72